KNN (K-Nearest Neighbors) Investigación

🔍 KNN (K-Nearest Neighbors): Investigación

> ⚠ Corrección importante sobre la tarea **KNN no es un modelo de agrupamiento (clustering).** Es un algoritmo de **aprendizaje supervisado** para **clasificación y regresión**. El algoritmo de agrupamiento que se confunde con él es K-Means (aprendizaje no supervisado). Las dos "K" significan cosas distintas. La sección 11 compara ambos para que este documento sirva sin importar cómo lo haya querido el maestro. Conviene confirmarlo con el maestro.

---

1. Introducción

K-Nearest Neighbors (KNN, "k vecinos más cercanos") es un algoritmo de Machine Learning que predice la etiqueta o el valor de una observación nueva a partir de las K observaciones más parecidas (las más cercanas) del conjunto de entrenamiento.

Su idea es la de un refrán: *"dime con quién andas y te diré quién eres"*. Si quieres saber de qué clase es un punto nuevo, miras a sus vecinos y decides por ellos.

Tres características lo definen:

Característica	Qué significa
Supervisado	Necesita datos con la respuesta correcta conocida (etiquetas o valores objetivo).
No paramétrico	No asume una forma matemática para los datos (como una recta o una distribución normal). La "forma" la dictan los datos.
Perezoso (lazy) / basado en instancias	No hay fase de entrenamiento real: solo guarda los datos. Todo el trabajo ocurre al predecir.

La documentación de scikit-learn lo describe como un método de aprendizaje "no generalizante": en lugar de construir un modelo interno, recuerda todos los datos de entrenamiento.

---

2. Historia y fundamentos teóricos

Año	Aporte	Autores
1951	Primer planteamiento de la regla del vecino más cercano en un reporte técnico de la USAF School of Aviation Medicine. Los autores nunca lo publicaron formalmente.	Fix y Hodges
1967	Demuestran que el error asintótico de la regla del vecino más cercano es como máximo el doble del error de Bayes.	Cover y Hart
1968	Extensión de la teoría de estimación con la regla del vecino más cercano.	Cover
1968	*Condensed Nearest Neighbor*: reducir el conjunto de entrenamiento guardando solo lo necesario.	Hart
1976	KNN ponderado por distancia: vecinos más cercanos pesan más.	Dudani
1999	Análisis de cuándo "vecino más cercano" deja de tener sentido en alta dimensión.	Beyer et al.

Resultado clave (Cover & Hart, 1967): con muchísimos datos, el error de la regla 1-NN nunca supera el doble del error óptimo posible (el error de Bayes). Cover lo resumió diciendo que la mitad de la información de clasificación está en el vecino más cercano. Es una garantía notable para un método tan simple.

El reporte de Fix y Hodges de 1951 fue reimpreso y comentado décadas después por Silverman y Jones (1989), lo que lo hizo accesible a la comunidad estadística.

---

3. ¿Cómo funciona? (algoritmo paso a paso)

Para predecir la clase de un punto nuevo **Q**:

1. **Elegir K** (cuántos vecinos consultar).
2. **Elegir una métrica de distancia** (por ejemplo, euclidiana).
3. **Calcular la distancia** de Q a **todos** los puntos de entrenamiento.
4. **Ordenar** las distancias de menor a mayor.
5. **Tomar los K más cercanos.**
6. **Decidir:**
   - *Clasificación:* **voto por mayoría** entre las clases de los K vecinos.
   - *Regresión:* **promedio** (o mediana) de los valores de los K vecinos.

Ejemplo numérico completo

Datos de entrenamiento con dos variables (x1, x2) y una clase (Rojo o Azul). Queremos clasificar **Q = (4, 4)** con distancia euclidiana:

```
d = √[(x1 − 4)² + (x2 − 4)²]
```

Punto	(x1, x2)	Clase	Distancia a Q
D	(5, 5)	Azul	√2 ≈ 1.41
C	(2, 3)	Rojo	√5 ≈ 2.24
A	(1, 2)	Rojo	√13 ≈ 3.61
E	(7, 7)	Azul	√18 ≈ 4.24
B	(2, 0)	Rojo	√20 ≈ 4.47
F	(6, 9)	Azul	√29 ≈ 5.39

Resultados según K:

K	Vecinos usados	Votos	Predicción
1	D	Azul 1	Azul
3	D, C, A	Rojo 2, Azul 1	Rojo
5	D, C, A, E, B	Rojo 3, Azul 2	Rojo

> Lección del ejemplo La predicción **cambió** al pasar de K=1 a K=3. K es un hiperparámetro que modifica el resultado y hay que elegirlo con cuidado (sección 5).

Versión ponderada (K=3, peso = 1/d):

- Azul (D): 1/1.41 ≈ 0.707
- Rojo (C y A): 1/2.24 + 1/3.61 ≈ 0.447 + 0.277 ≈ 0.724

El resultado es casi un empate (Rojo gana por poco). Esto ilustra que el ponderado puede cambiar o endurecer decisiones fronterizas.

Ejemplo de regresión: si los tres vecinos más cercanos de Q tienen valores Y = 10, 12 y 20, la predicción es `(10 + 12 + 20) / 3 = 14`.

---

4. Métricas de distancia

La métrica define qué significa "cercano". Cambiarla cambia los vecinos y, por tanto, el modelo.

Métrica	Fórmula (resumen)	Cuándo usarla
Euclidiana	$\sqrt{\Sigma(x_i - y_i)^2}$	Opción por defecto con variables numéricas continuas y escaladas.
Manhattan	$\Sigma\|x_i - y_i\|$	Más robusta a valores atípicos; útil en dimensiones algo más altas.
Minkowski	$(\Sigma\|x_i - y_i\|^p)^{(1/p)}$	Generalización: p=1 es Manhattan, p=2 es Euclidiana.
Chebyshev	$\max\|x_i - y_i\|$	Cuando importa la mayor diferencia en una sola variable.
Coseno	$1 - \cos(\theta)$ entre vectores	Texto y vectores de alta dimensión donde importa la dirección, no la magnitud.
Hamming	Número de posiciones distintas	Variables categóricas o binarias.

---

5. ¿Cómo elegir K?

K es el principal hiperparámetro de KNN y controla el balance sesgo-varianza.

	K pequeño (ej. 1)	K grande (ej. 50)
Frontera de decisión	Muy irregular, se pega a cada punto	Suave, más general
Riesgo	Sobreajuste (overfitting): sensible al ruido	Subajuste (underfitting): ignora patrones locales
Varianza	Alta	Baja
Sesgo	Bajo	Alto

Reglas prácticas:

- Usar **K impar** en clasificación binaria para evitar empates.
- La heurística $K \approx \sqrt{n}$ (n = número de muestras) es solo un **punto de partida**, no una regla.
- La forma correcta es **probar varios valores con Validación Cruzada (Cross-Validation)** y quedarse con el que mejor desempeño dé.
- **Nunca** elegir K mirando el conjunto de prueba (test): eso es Data Leakage.

---

6. Preparación de datos (crítico en KNN)

Como KNN vive de las distancias, **la calidad de los datos pesa más que en casi cualquier otro modelo**.

1. **Escalado obligatorio.** Si una variable está en miles (salario) y otra en unidades (edad), la primera dominará la distancia. Ver Transformación y Normalización. Opciones: `StandardScaler` (z-score) o `MinMaxScaler`.
2. **Valores faltantes.** KNN no tolera NaN en el cálculo de distancias: hay que imputar antes. Ver Data Cleaning.
3. **Variables categóricas.** Codificar (one-hot) o usar una métrica adecuada como Hamming.
4. **Selección de variables.** Las variables irrelevantes agregan "ruido de distancia". Quitarlas mejora mucho el modelo.
5. **Evitar fuga de datos.** Ajustar el escalador **solo con el train** y aplicarlo al test. La forma segura es un `Pipeline` (sección 10).

> 🔥 **El error más común con KNN: no escalar. Un KNN sin escalar suele dar resultados engañosos, no solo malos.**

---

7. Variantes

Variante	Idea	Detalle
KNN uniforme	Todos los vecinos votan igual	Es el comportamiento por defecto.
KNN ponderado por distancia	Más cerca, más peso	Propuesto por Dudani (1976). Reduce el efecto de vecinos lejanos.
Radius Neighbors	En vez de K vecinos, todos los puntos dentro de un radio r	Útil cuando la densidad de datos es muy desigual.
Condensed NN	Reducir el conjunto de entrenamiento a los puntos esenciales	Hart (1968). Menos memoria y predicción más rápida.
KNN para regresión	Promedia los valores de los K vecinos	`KNeighborsRegressor`.
KNN imputer	Rellena valores faltantes con los de vecinos	Aplicación de KNN a Data Cleaning.

---

8. Complejidad y eficiencia

KNN es barato de "entrenar" pero **caro de usar**.

Aspecto	Costo
Entrenamiento	Prácticamente nulo (solo guarda los datos).
Memoria	O(n · d), guarda todo el dataset (n muestras, d dimensiones).
Predicción por fuerza bruta	O(n · d) por cada consulta.

Para acelerar la búsqueda de vecinos se usan estructuras de datos:

Estructura	Idea	Conviene cuando
Fuerza bruta (brute force)	Compara contra todos los puntos	Pocos datos o muy alta dimensión.
KD-Tree	Divide el espacio por ejes de forma recursiva	Baja dimensión (pocas variables).
Ball Tree	Agrupa puntos en hiperesferas anidadas	Se diseñó para corregir la ineficiencia del KD-Tree en dimensiones mayores.

En scikit-learn el parámetro `algorithm` acepta `auto`, `kd_tree`, `ball_tree` y `brute`. Con `auto` el sistema elige según los datos.

---

9. La maldición de la dimensionalidad

Cuando hay muchas variables, los datos se vuelven dispersos y las distancias entre puntos tienden a parecerse: el vecino más cercano y el más lejano quedan casi a la misma distancia. Entonces "cercano" pierde significado. Beyer et al. (1999) estudiaron formalmente esta pregunta: *¿cuándo tiene sentido el vecino más cercano?*

Matiz honesto: ese trabajo no afirma que la búsqueda de vecinos en alta dimensión nunca tenga sentido; muestra que bajo ciertas condiciones de los datos el concepto se degrada.

Cómo mitigarlo:

- Selección de variables (quedarse con las relevantes).
- Reducción de dimensionalidad (por ejemplo, PCA).
- Métricas más adecuadas (coseno, Manhattan).
- Conseguir más datos.

---

10. Implementación en Python (scikit-learn)

Clasificación con búsqueda del mejor K

```python
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier()),
])

params = {
    "knn__n_neighbors": [1, 3, 5, 7, 9, 11, 15],
    "knn__weights": ["uniform", "distance"],
    "knn__metric": ["euclidean", "manhattan"],
}

grid = GridSearchCV(pipe, params, cv=5, scoring="f1_macro")
grid.fit(X_train, y_train)

print(grid.best_params_)
print(classification_report(y_test, grid.predict(X_test)))
```

Qué hace cada parte:

- `Pipeline` encadena escalado y modelo; el escalador se ajusta **solo con el train** de cada fold, así que no hay fuga.
- `GridSearchCV` prueba combinaciones de K, pesos y métrica con validación cruzada de 5 folds.
- `stratify=y` mantiene la proporción de clases en train y test.
- `classification_report` evalúa en datos que el modelo nunca vio.

Regresión

```python
from sklearn.neighbors import KNeighborsRegressor

reg = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsRegressor(n_neighbors=5, weights="distance")),
])
```

```
# reg.fit(X_train, y_train); reg.predict(X_test)
```

Para evaluar se usan métricas de regresión (RMSE, MAE, R^2). Ver Implementación y Evaluación si tienes esa nota.

---

11. KNN vs. K-Means: la confusión de las dos "K"

Aspecto	KNN	K-Means
Tipo de aprendizaje	Supervisado	No supervisado
¿Necesita etiquetas?	Sí	No
Problema que resuelve	Clasificación y regresión	Agrupamiento (clustering)
Significado de K	Cuántos vecinos se consultan	Cuántos grupos se forman
¿Entrena?	No (solo guarda los datos)	Sí (itera moviendo centroides)
Salida	Una etiqueta o valor para una observación nueva	Un grupo asignado a cada observación, más los centroides
Evaluación	Contra la respuesta correcta (accuracy, F1, RMSE)	No hay respuesta correcta: codo, silueta, utilidad para el negocio
Sensible a la escala	Sí	Sí

> ✎ **Dato para no confundirse más `NearestNeighbors` de scikit-learn es una herramienta de búsqueda de vecinos no supervisada. No es lo mismo que el clasificador `KNeighborsClassifier`. Ni uno ni otro agrupan como lo hace K-Means.**

Si el maestro pidió realmente un modelo de agrupamiento, el tema correcto es K-Means (también existen DBSCAN, jerárquico, etc.). Si pidió KNN, este documento es el indicado.

---

12. Ventajas y desventajas

✅ Ventajas	❌ Desventajas
Muy fácil de entender e implementar	Predicción lenta con muchos datos
Sin fase de entrenamiento	Consume mucha memoria (guarda todo)

✅ Ventajas	❌ Desventajas
No asume forma de los datos (no paramétrico)	Muy sensible a la escala de las variables
Sirve para clasificación y regresión	Sufre en alta dimensión
Captura fronteras de decisión complejas	Sensible a ruido y variables irrelevantes (con K bajo)
Buen punto de partida (baseline)	Se comporta mal con clases muy desbalanceadas sin ajustes

---

13. Aplicaciones

- **Sistemas de recomendación:** usuarios o productos "parecidos" (filtrado colaborativo basado en vecinos).
- **Imputación de valores faltantes:** rellenar con datos de observaciones similares.
- **Reconocimiento de patrones:** dígitos manuscritos, imágenes sencillas.
- **Medicina:** clasificar casos según pacientes con perfiles similares.
- **Detección de anomalías:** puntos lejos de sus vecinos (por ejemplo, LOF, Local Outlier Factor).
- **Línea base (baseline):** modelo simple contra el cual comparar algo más complejo como Random Forest.

---

14. Conclusiones

1. KNN es un algoritmo **supervisado y no paramétrico** que predice a partir de los K ejemplos más parecidos; **no es de agrupamiento**.
2. Su simplicidad es real, pero el resultado depende de tres decisiones: **K, métrica de distancia y escalado**.
3. K se elige con **validación cruzada**, nunca con el conjunto de prueba.
4. Su costo está en la **predicción**, no en el entrenamiento, y se degrada en **alta dimensión**.
5. La confusión con K-Means es común; la clave es recordar que en KNN la K cuenta **vecinos** y en K-Means cuenta **grupos**.

---

15. Referencias bibliográficas

1. Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory, IT-13*(1), 21–27. https://ieeexplore.ieee.org/document/1053964

2. Fix, E., & Hodges, J. L., Jr. (1951). *Discriminatory analysis, nonparametric discrimination: Consistency properties* (Project 21-49-004, Report 4). USAF School of Aviation Medicine, Randolph Field, TX. https://apps.dtic.mil/sti/pdfs/ADA800276.pdf
3. Beyer, K., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). When is "nearest neighbor" meaningful? En *Database Theory — ICDT'99* (LNCS 1540, pp. 217–235). Springer. https://doi.org/10.1007/3-540-49257-7_15
4. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2.ª ed.). Springer. https://www.statlearning.com (KNN en las secciones 3.5 y 4.7.6).
5. scikit-learn developers (s. f.). *Nearest Neighbors* (user guide). https://scikit-learn.org/stable/modules/neighbors.html
6. Cover, T. M. (1968). Estimation by the nearest neighbor rule. *IEEE Transactions on Information Theory, IT-14*(1), 50–55.
7. Hart, P. E. (1968). The condensed nearest neighbor rule. *IEEE Transactions on Information Theory, IT-14*, 515–516.
8. Dudani, S. A. (1976). The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics, SMC-6*, 325–327.
9. Silverman, B. W., & Jones, M. C. (1989). E. Fix and J. L. Hodges (1951): an important contribution to nonparametric discriminant analysis and density estimation. *International Statistical Review, 57*(3), 233–247.

📝 Notas relacionadas

- K-Means
- Árboles de Decisión
- Random Forest
- Regresión Lineal
- Regresión Logística
- Machine Learning

🔗 Conceptos relacionados

- Transformación y Normalización
- Validación Cruzada (Cross-Validation)
- Data Leakage
- Data Cleaning